

分布式系统基础

Slides采用了南洋理工大学

Tang Xueyan和Anwitaman Datta

博士讲稿的材料

Fundamentals – Foundation

Characterization of Distributed Systems & System Models

Outline

- What is a Distributed System (DS)?
- Fundamental characteristics of DS
- Main motivation of DS – resource sharing (the Web example)
- What are the issues and problems in DS?
- Architecture models
- Fundamental models

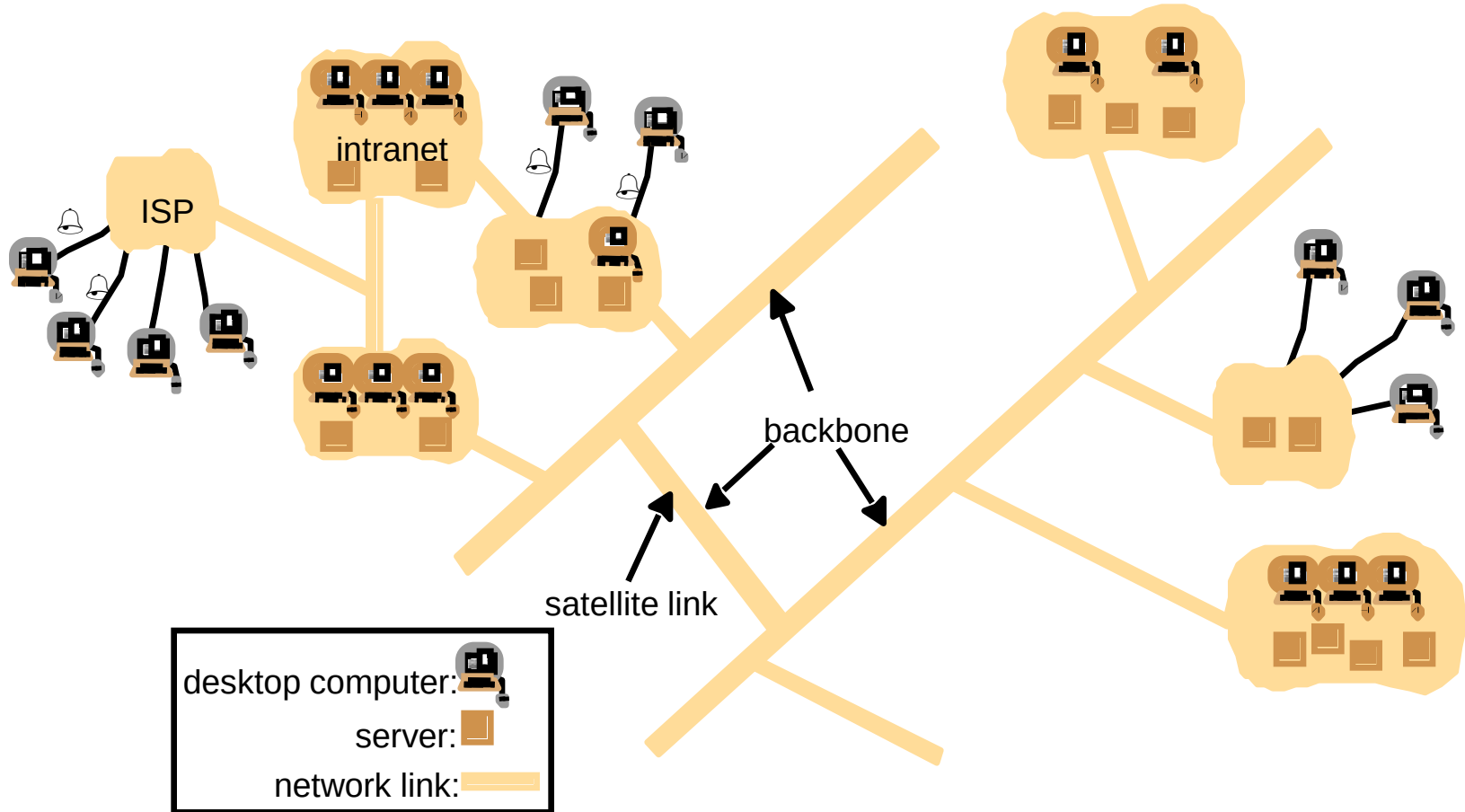
What is a Distributed System?

- Human society evolves into a distributed system
- Structures/protocols that evolved, e.g.,
 - We live in manageable, autonomous units – families
 - We work in organizations; some members become specialized
 - To achieve different purposes, we cooperate and interact with each other in certain ways
 - Phone calls, sending emails, shouting in stock market etc.
 - We use clocks and landmarks to synchronize with each other (e.g. attending classes in lecture theatres)

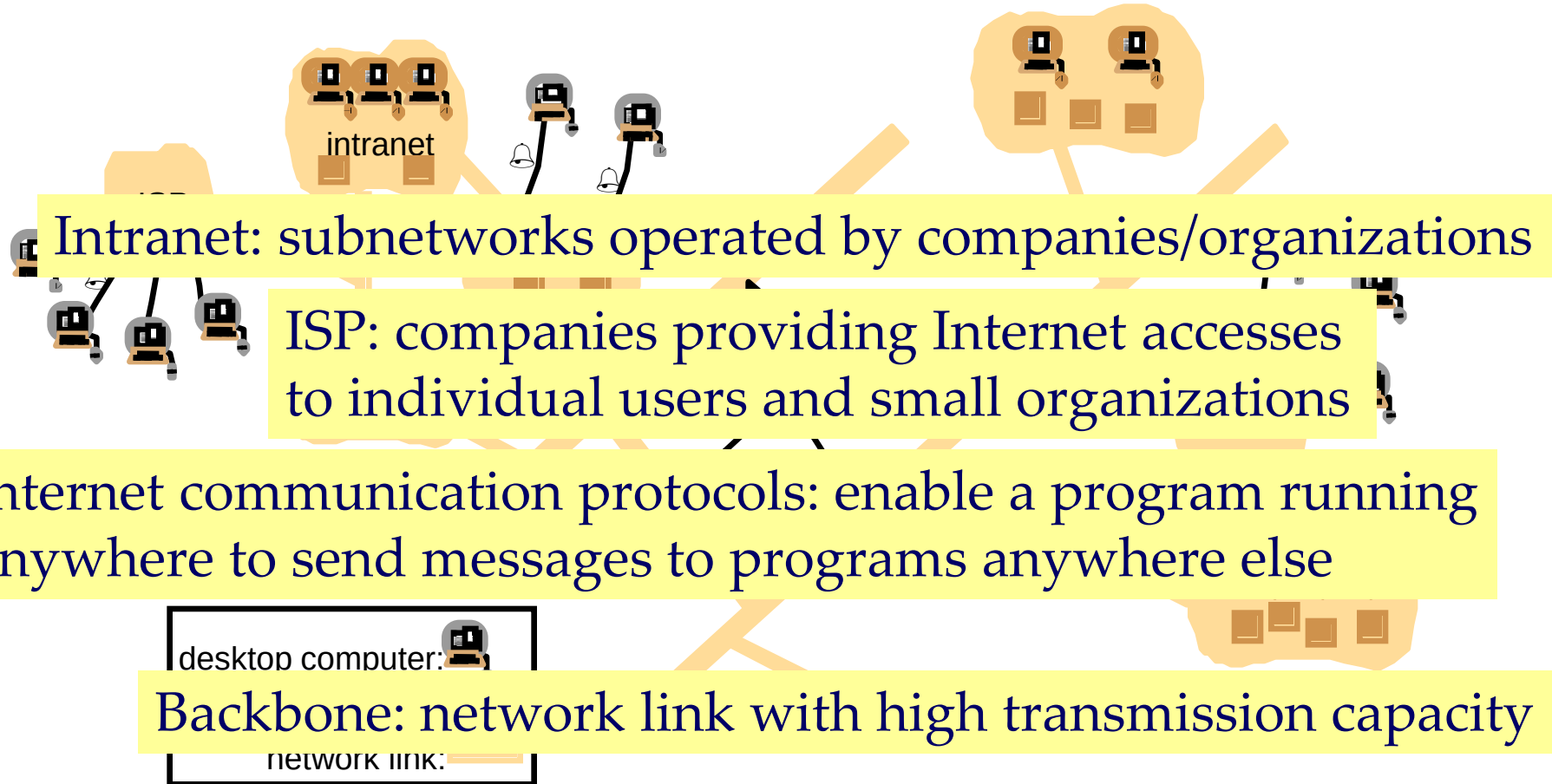
What is a Distributed System?

- Computers/devices also evolve into distributed systems
- Enabled by Internet, campus networks, home networks, corporate networks, mobile phone networks, ...
- Definition
 - A set of **networked computers**
 - that communicate and coordinate their actions only by **passing messages**

Example: The Internet



Example: The Internet



Internet: **a very large distributed system**, enabling users to make use of services like WWW, email & file transfer from everywhere

Outline

- What is a Distributed System (DS)?
- **Fundamental characteristics of DS**
- Main motivation of DS – resource sharing (the Web example)
- What are the issues and problems in DS?
- Architecture models
- Fundamental models

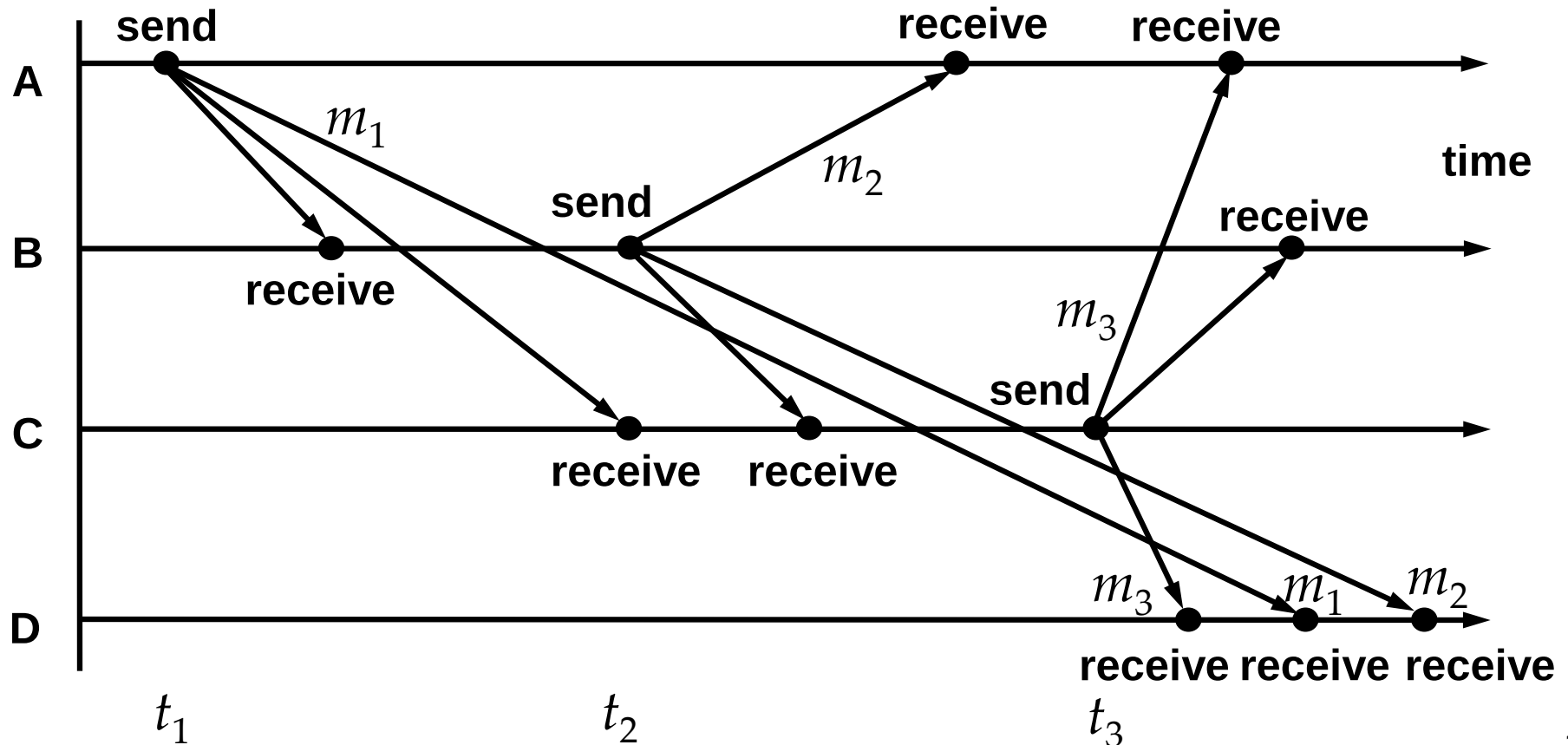
Fundamental Characteristics of DS

- Concurrency: **concurrent program execution**
 - Higher capacity → Possibly higher performance
 - System capacity can be increased by adding more resources (e.g., computers) to the network
 - Need to coordinate concurrently executing programs

Fundamental Characteristics of DS

- Loosely coupled:

- no global clock** (difficult to synchronize perfectly, lack of certainty, difficult to design correct algorithms)



Fundamental Characteristics of DS

- Loosely coupled (cont'd)
 - **no global shared memory** (programs interact by passing messages → to reduce communication in design)
 - vs. **tightly coupled**: coordinate through global memory, synchronization through global clock (e.g., a multi-core processor)

Fundamental Characteristics of DS

- Independent failures: any computer (including its software/hardware components) and subnetwork can **fail at anytime**
 - Detection may be difficult or even impossible
 - However, distributed systems can be made **more fault-tolerant** than stand-alone systems if properly designed

Outline

- What is a Distributed System (DS)?
- Fundamental characteristics of DS
- Main motivation of DS – resource sharing (the Web example)
- What are the issues and problems in DS?
- Architecture models
- Fundamental models

Distributed Services & Resource Sharing

- Resources may be in the form of
 - Information (e.g., text/picture/audio/video, news, email)
 - Hardware (e.g., computing power, storage, cameras, printers)
 - Software (e.g., system software, application programs)
- Benefits: save cost and man-power

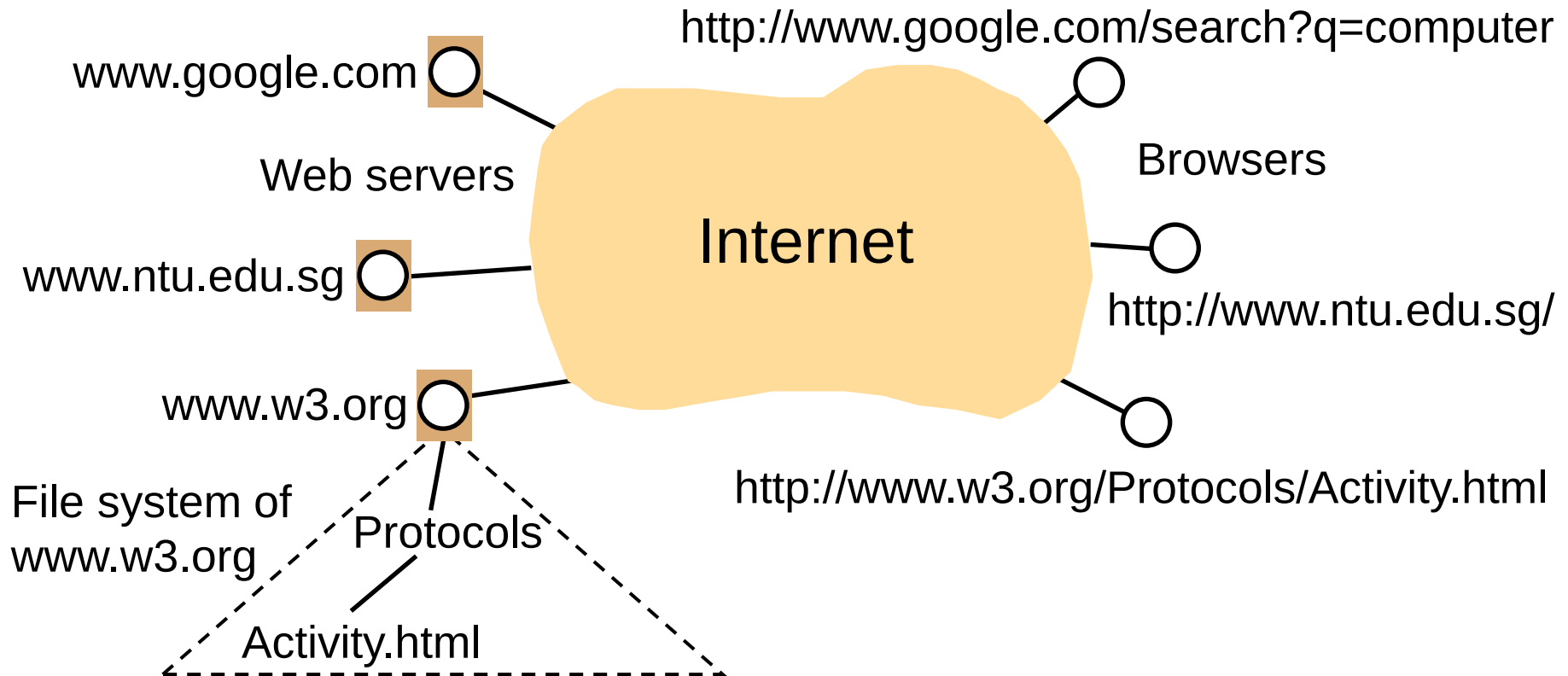
Distributed Services & Resource Sharing

- How are resources shared? – via services
 - Service: a distinct part of a computer system **providing accesses to the managed resources**
 - e.g., read/write/delete operations on files
 - **Server**: the process managing resources (e.g., web server)
 - **Client**: the process requesting to access resources (e.g., web browser)
 - The server accepts requests from clients, performs services, and responds to clients
 - Requests and replies are sent in the form of messages
 - **Remote invocation: a complete interaction between a client and a server**, from the point when the client sends its request to the point when it receives the server's response

How Does Resource Sharing Work?

- **Content and format of resources:** what are the resources and how to make use of them?
- **Naming & addressing:** how to name the service/resource and locate it in the network?
- **Communication infrastructures:** how does the client communicate with the server?

Example: The Web



World Wide Web

- **Content and format of resources**

- HTML (Hypertext Markup Language): specify the contents and layout of pages for web browsers to render

- An example piece of HTML text:

`<IMG SRC = "http://www.ntu.edu.sg/sce/images/pic.jpg">`

`<P> Welcome to Nanyang Technological University!`

`<A HREF = "http://www.ntu.edu.sg/sce/"> SCE </A>`

World Wide Web

■ Naming & Addressing

- URL (Uniform Resource Locator): to name documents and other resources stored on the web
- General format: **scheme: scheme-specific-location**
 - Scheme declares the type of URL
 - <http://www.w3.org>
 - <ftp://ftp.downloadIt.com/software/aprog.exe>
 - <telnet://delta.ntu.edu.sg>
- Format of HTTP URLs
 - **[http://servername\[:port\]\[/pathNameOnServer\]\[?arguments\]](http://servername[:port][/pathNameOnServer][?arguments])**
 - <http://www.ntu.edu.sg>
 - <http://www.w3.org/Protocols/Activity.html>
 - <http://www.google.com/search?q=computer>

World Wide Web

- **Naming & Addressing (cont'd)**
 - DNS (Domain Name System): to translate domain names of hosts to their Internet (IP) addresses
- **Communication infrastructures:** relies on TCP/IP +
 - HTTP (Hypertext Transfer Protocol): standard rules for request-reply interactions between browsers and servers
 - The browser sends a request message (containing the URL) to the server who sends back the requested resource in a reply message

Outline

- What is a Distributed System (DS)?
- Fundamental characteristics of DS
- Main motivation of DS – resource sharing (the Web example)
- What are the issues and problems in DS?
- Architecture models
- Fundamental models

Challenges of Distributed Systems

- Heterogeneity – hardware and software components are not identical
 - Networks (Ethernet, ISDN connections, modem connections, wireless LAN, infra-red, bluetooth, etc.)
 - Masked by all computers using the Internet protocols to communicate with one another
 - Computer hardware (supercomputers, desktop PCs, laptops, tablet computers, wearable devices, etc.)
 - For example, two alternatives for byte orderings: big-endian and little-endian
 - How to exchange data between programs running on different hardware?

Challenges of Distributed Systems

- Heterogeneity (cont'd)
 - Operating systems (Windows, Unix, etc.)
 - Different application programming interfaces
 - e.g., different system calls for exchanging messages in UNIX and Windows
 - Programming languages
 - e.g., different representations for characters, arrays ...
 - How can programs written in one language communicate with those in another language?

Challenges of Distributed Systems

- Scalability: to remain effective when there is a significant increase in number of resources & users
 - Computers and web servers in the Internet (according to surveys by Internet Systems Consortium and Netcraft)

Date	Computers	Web Servers
July, 1993	1,776,000	130
July, 1995	6,642,000	23,500
July, 1999	56,218,000	6,598,697
July, 2002	162,128,493	37,235,470
July, 2006	439,286,364	88,166,395
July, 2010	768,913,036	205,714,253

Challenges of Distributed Systems

■ Scalability (cont'd)

- Approaches to handle increasing demand
 - Caching and replication of data
 - Deploy multiple servers
- Control the cost of physical resources
 - At most $O(n)$ resources to support $O(n)$ users
- Control the performance loss
 - e.g., $O(\log n)$ time to access a set of $O(n)$ data
- Prevent software resources from running out
 - e.g., length of Internet (IP) addresses
- Avoid performance bottlenecks
 - Prefer decentralized design to centralized design

Challenges of Distributed Systems

■ Failure handling

- Detecting (e.g., checksums to detect corrupted messages)
- Masking (e.g., retransmit messages when they are lost)
- Tolerating (**use redundancy to achieve fault tolerance**, e.g., store the same files on two disks)
- Recovery (“roll back” when a server crashes, not to leave data in an inconsistent state)

■ Self-learning: more challenges of distributed systems

- Reference book A, Section 1.5

Outline

- What is a Distributed System (DS)?
- Fundamental characteristics of DS
- Main motivation of DS – resource sharing (the Web example)
- What are the issues and problems in DS?
- **Architecture models**
- Fundamental models

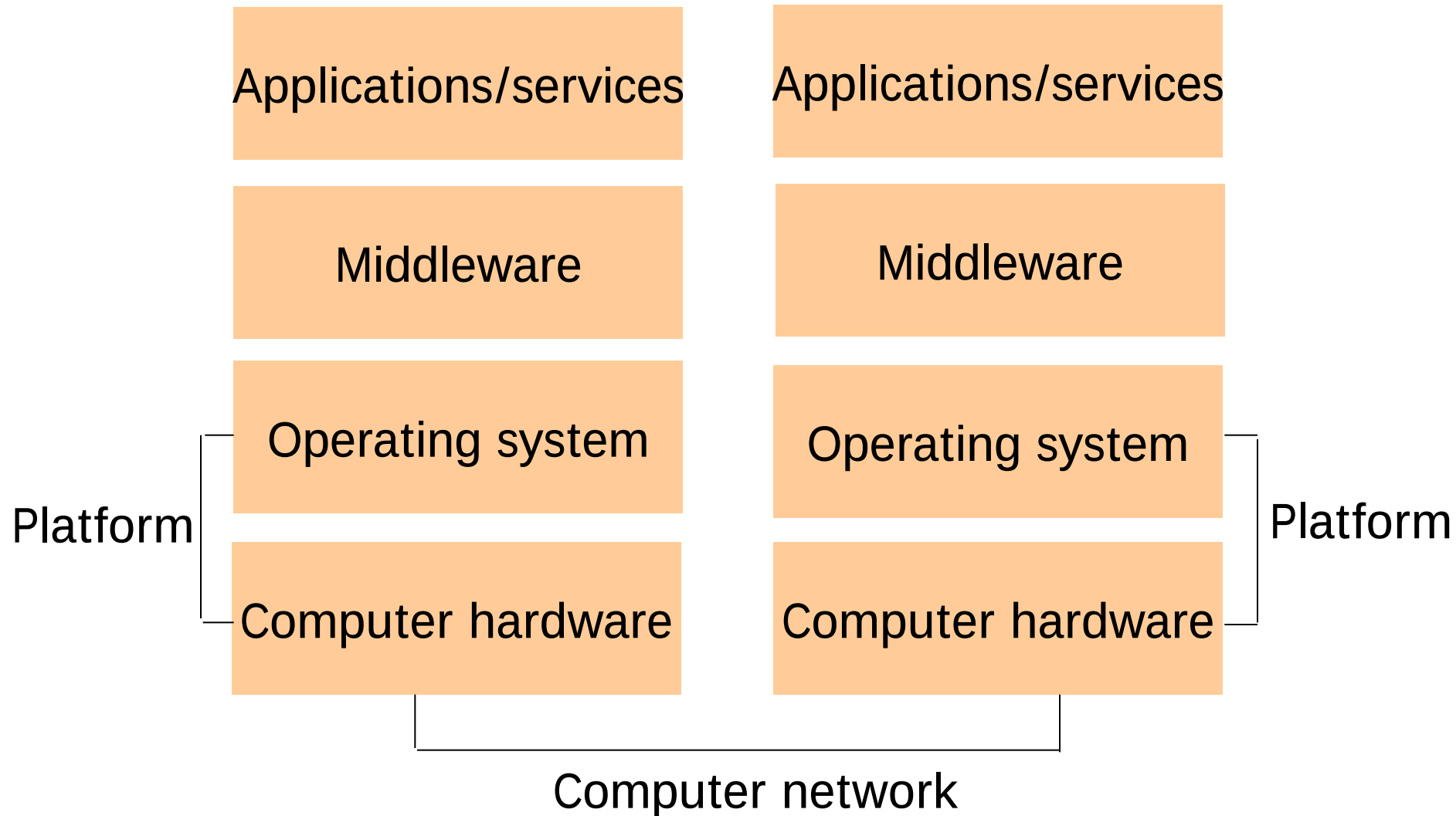
Models

- An abstract, simplified but consistent description
 - Example: Von Neumann's model for computers
- Architectural models:
 - Identify components; describe functions; placement of components; relationship between components (e.g., how do they interact with one another?)
- Fundamental models:
 - More formal description of **common and intrinsic properties** in architectural models
 - Deal with correctness, reliability and security
 - Make/verify exact claims by logical analysis and mathematical proof

Architectural Models

- **Software and hardware layers (“vertical”)**
 - Structuring of software as layers/modules in terms of services offered and requested between processes located in the same or different computers
- **System architectures (“horizontal”)**
 - Client-server model
 - Peer-to-peer model

Software & hardware service layers in DS



Platform

- Lowest two levels (hardware and OS) → platform
 - Bring the system's programming interface up to a level that facilitates communication and coordination between processes
 - e.g., Intel x86/Windows, Sun SPARC/SunOS

Middleware

- A software layer to mask heterogeneity and provide a convenient programming model
 - Heterogeneity in underlying network protocols, hardware, operating systems, programming languages
- Provide **generic services** to applications, e.g.,
 - Naming: e.g., add/search name-to-address mappings
 - Security: e.g., authentication
 - Persistent storage: e.g., store and retrieve objects
 - Event notifications: e.g., friend has come online
- Example:
 - Sun RPC, Java RMI
 - CORBA (Common Object Request Broker Architecture)

Architectural Models (Revisit)

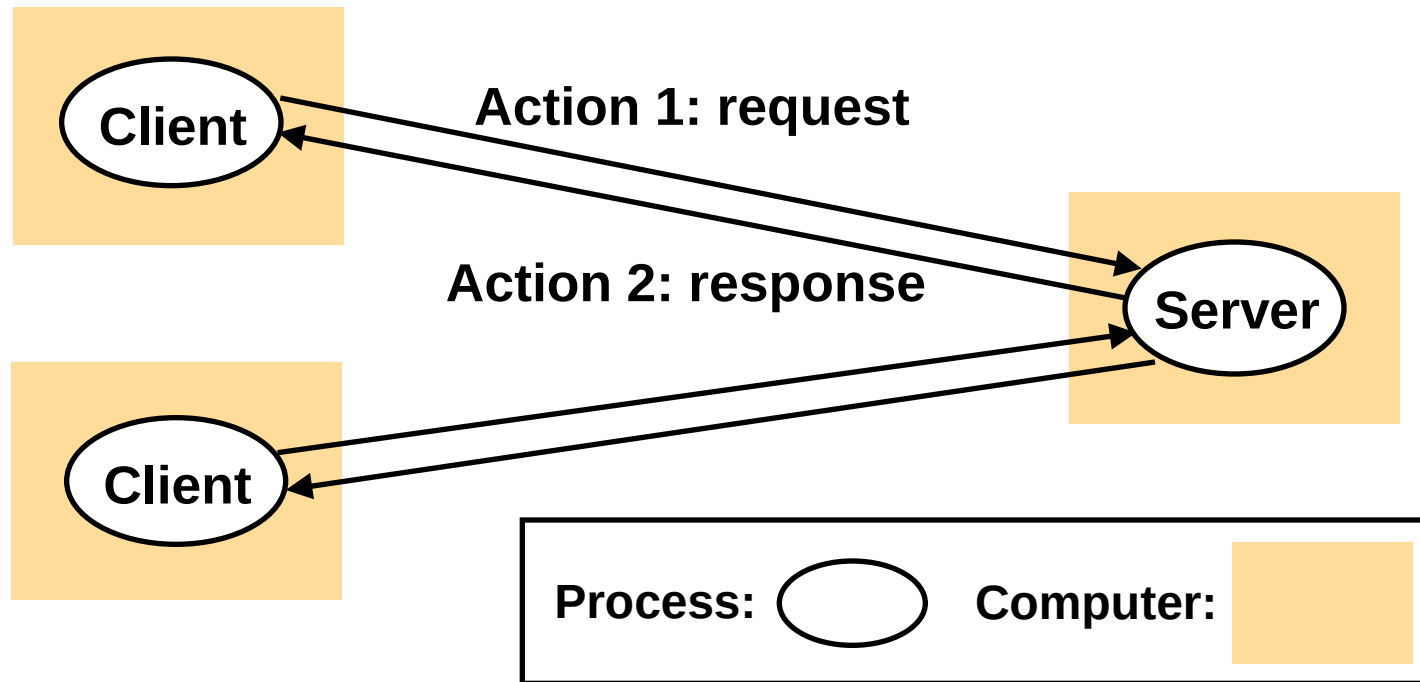
- Software and hardware layers (“vertical”)
- System architectures (“horizontal”)
 - Client-server model
 - Peer-to-peer model

Client-Server Model

- Server: processes that **manage resources to provide a service**, they accept requests from clients and respond
- Client: processes that **use/access the service**
- Client-server is a simple yet useful model for developing distributed systems

Simple Client-Server Model

- Simple request-response interactions



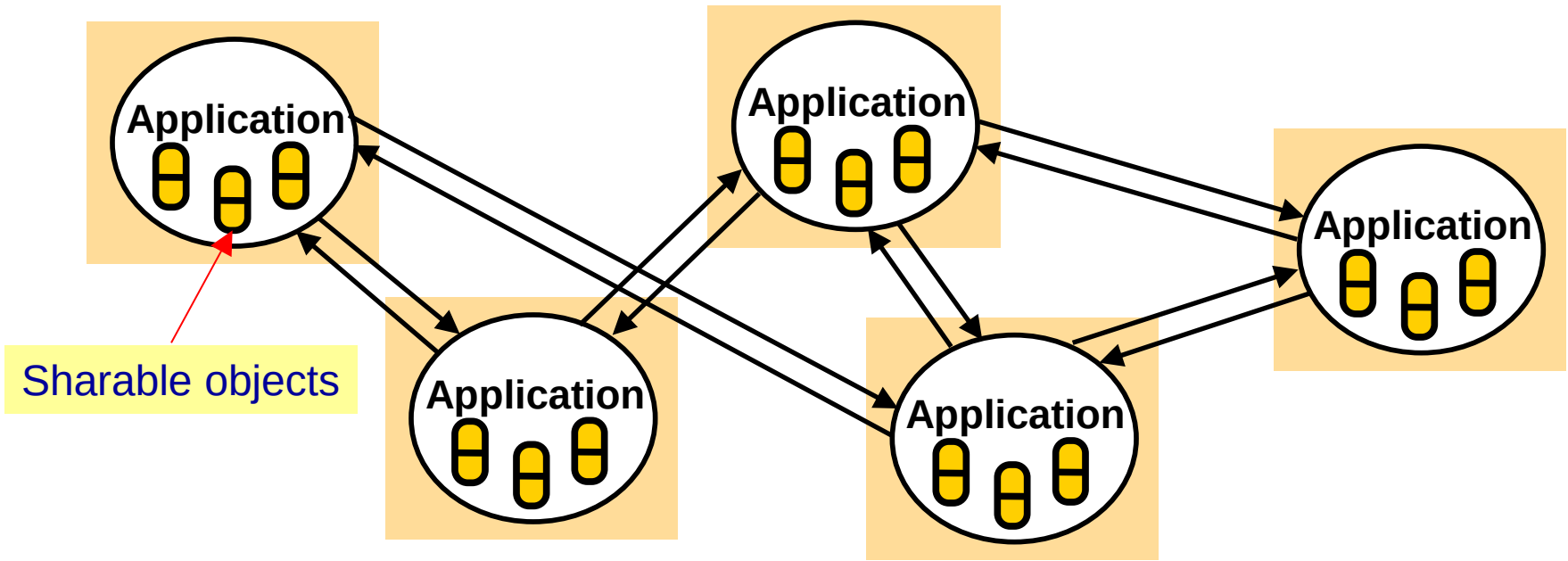
- N clients accessing the same service provided by a server: N-to-1

Variation of Client-Server Model

More generally:

- Services may be provided by multiple servers
- M servers working together for N clients
- Division of work among M servers:
 - partitioning: function (cooperative servers) or data
 - e.g., each server manages its own set of resources, `www.cnn.com`, `money.cnn.com`, `sports.cnn.com`
 - replication: function (identical servers) or data
 - e.g., servers manage the same set of resources, `www.google.com`, `www.google.com.sg`, `www.google.com.hk`

Peer-to-Peer Model



- All processes play **similar roles**, they interact cooperatively as peers to perform distributed computation
- No distinction between clients and servers
- e.g., chatting, whiteboard, p2p file sharing
- Peer-to-peer architecture is substantially more complex than client-server architecture

Outline

- What is a Distributed System (DS)?
- Fundamental characteristics of DS
- Main motivation of DS – resource sharing (the Web example)
- What are the issues and problems in DS?
- Architecture models
- Fundamental models

Fundamental Models

- Revisit: more formal description of **common and intrinsic properties** in architectural models
- Interaction Model
 - Assumptions about behaviors in terms of time
- Failure Model
 - Define and classify faults
 - Basis for designing systems to tolerate faults

Interaction Model

- **Synchronous distributed system:** assume upper/lower bounds on
 - processing time (of each step of a program)
 - transmission time (communication performance is often a limiting factor for the accuracy of coordination)
 - clock drift rate (computer clocks drift away from perfect time)
- Synchronous distributed systems can be built (e.g., reserve resources if requirements are known), but difficult
- Allow us to **infer properties** → see next slide

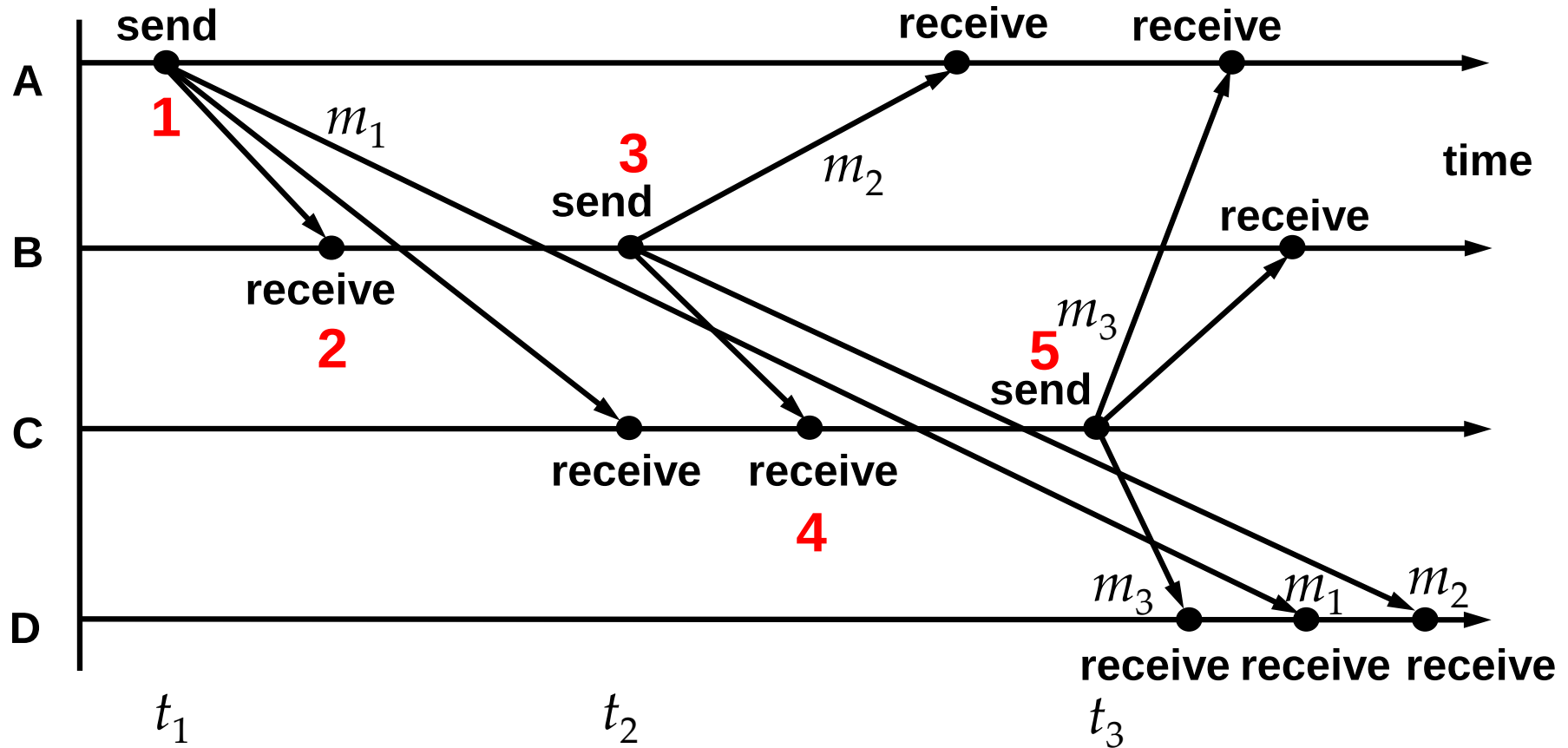
Interaction Model

- Allow us to **infer properties** such as:
 - Use timeouts to detect failures
 - Assume transmission time is in $[t_{\min}, t_{\max}]$, computer 1 sends a message to computer 2, then waits $t_{\min}$ before starting a program, computer 2 starts a program on receiving the message, then the program on computer 2 must be within a lag of $t_{\max} - t_{\min}$ from that on computer 1

Interaction Model

- **Asynchronous distributed system:** assume **no bounds** on
 - processing time (of each step of a program)
 - transmission time
 - clock drift rate
- Asynchronous distributed systems exactly model the Internet due to sharing of processors and networks
- Does **NOT** allow us to infer properties such as:
 - Using timeouts to detect failures
- But does allow us to infer other useful properties
→ see next slide

Example: Ordering of Events



- For any message m , event $Send(m)$ must **precede** $Receive(m)$
- Replies are sent **after** receiving messages
- Therefore, it can be inferred that $Send(m_1)$ **precedes** $Send(m_3)$

Failure Model

- Define the ways in which failures may occur
- **Omission Failures:** a process or communication channel fails to perform actions it is supposed to do
- Process omission failure – **crash:** process halts and will not execute any further steps of its program
 - Detecting crashes is possible in synchronous systems, but is difficult in asynchronous systems
- Communication omission failure – a message sent by one process is not delivered to the other process

The Generals' Paradox

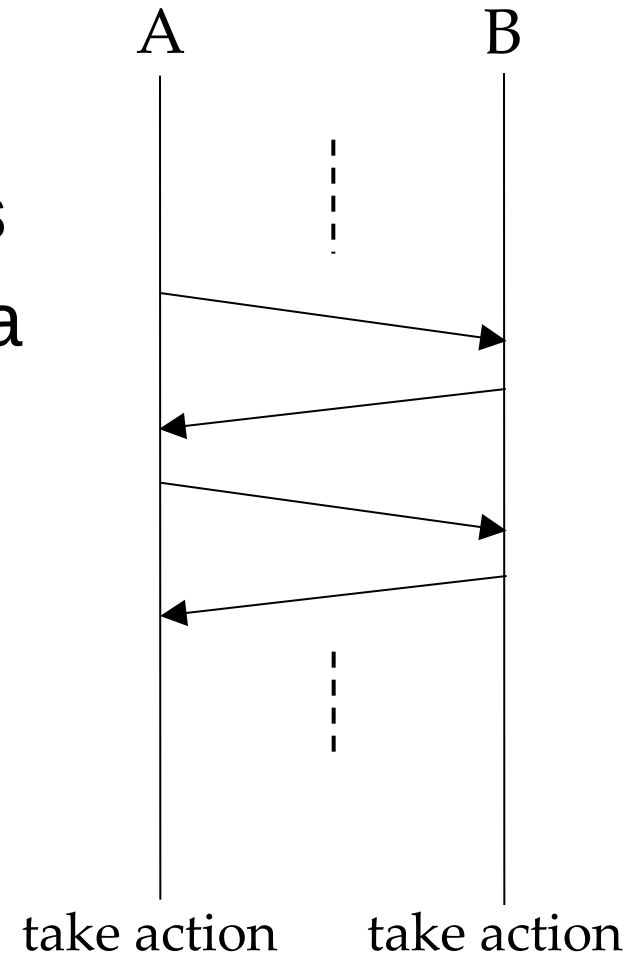
- Let's look at an example to illustrate the effects of failures on the design of a distributed system
- Problem
 - Two Pepperland army divisions: "Apple" & "Orange"
 - Their enemies "Blue Meanies" are located in between
 - Suppose "Blue Meanies" can capture the messages sent between "Apple" and "Orange" and prevent them from arriving at the destination
 - Can we design a protocol for "Apple" and "Orange" to consistently decide to both charge at the "Blue Meanies" or both surrender?
- It is impossible to design such a protocol

Formal Definition of the Problem

- Two processes A and B communicate by passing messages on a bidirectional channel
- Neither process can fail
- But there exist communication omission failures
 - A message sent from either process may be lost in the channel before reaching the other process
- Each process wants to select and take one of two actions x and y
- Can we devise a protocol for both processes to agree and take the same action?
- This problem has no solution – how to prove it?

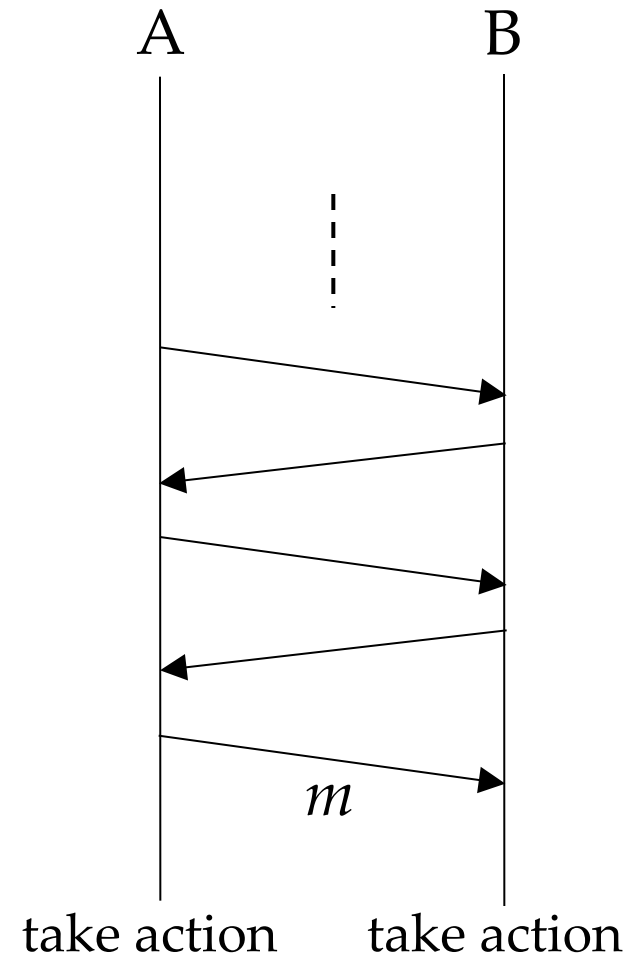
Proof

- Any protocol is equivalent to one in which A and B exchange messages in rounds
- Our approach: if there exists a protocol to solve the problem
→ derive a contradiction



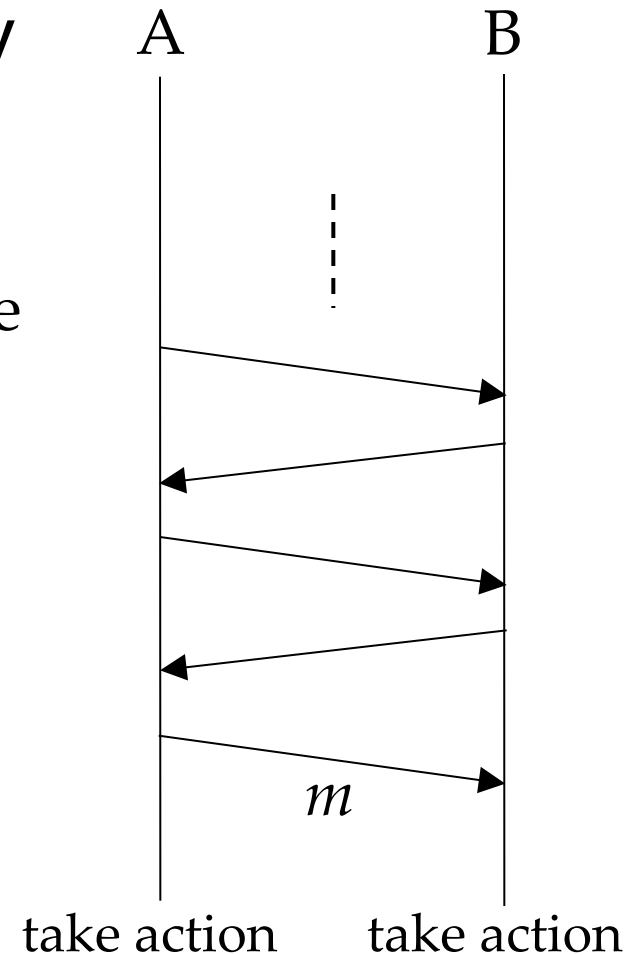
Proof

- Among all solutions, consider the protocol that solves the problem using the fewest message exchanges
 - No protocol solving the problem uses fewer message exchanges
- Without loss of generality, assume in this protocol, the last message m is sent by A



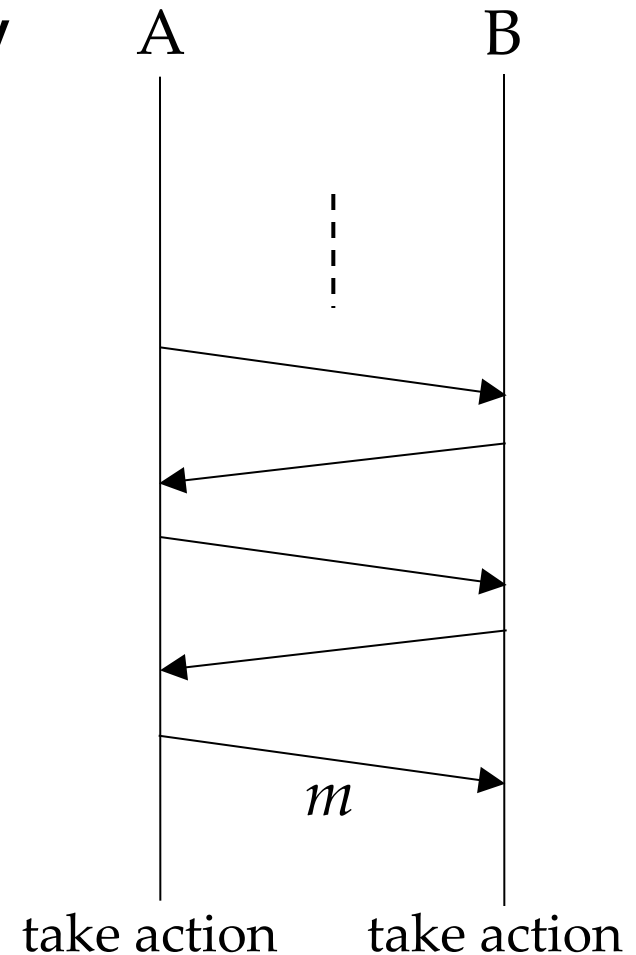
Proof

- The action ultimately taken by A cannot depend on whether m is received by B
 - communication-omission failure
→ m might be lost
 - m is the last message → B does not send any message after m
→ the receipt of m can never be learned by A
- So, A's choice of x or y does not depend on m



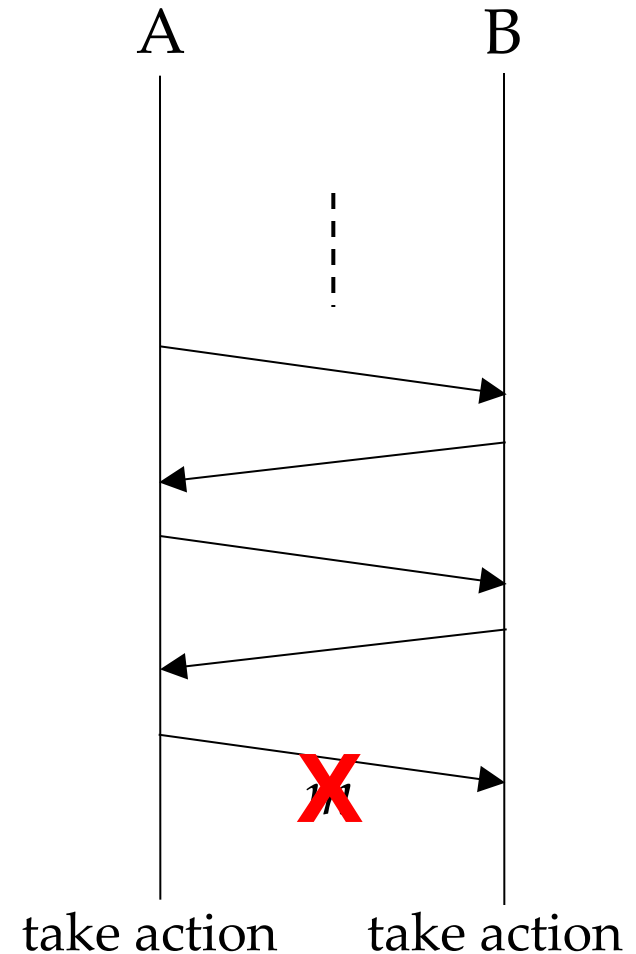
Proof

- Similarly, the action ultimately taken by B cannot depend on whether m is received by B
 - correct solution $\rightarrow$ A and B always take the same action
 - Since A's choice of action does not depend on m , B must make the same choice of action regardless of whether m is lost or not
- So, B's choice of x or y does not depend on m



Proof

- The actions chosen by A and B do not depend on m
- Therefore, m is unnecessary
→ we can discard it
- Now, we get a new protocol with one fewer message exchange → contradiction



Failure Model

- Arbitrary (Byzantine) Failures

- Do not simply fail but cause more trouble
- Arbitrarily omit intended processing steps and take **unintended** processing steps
- Processes, e.g.,
 - A process may set wrong values to its data items
 - It may return a wrong result in response to an invocation
- Communication, e.g.,
 - Corrupted messages may arrive at the destination
 - A message may be delivered more than once

Summary

- Distributed systems
 - A set of networked computers
 - communicate only by passing messages
- Fundamental characteristics of distributed systems: concurrency, loosely coupled, independent failures
- Resource sharing is the main motivation for constructing distributed systems
 - Typically, resources are managed by servers and accessed by clients

Summary

- Architectural models:
 - client-server & peer-to-peer
- Interaction models: assumptions about behaviors in terms of time
 - synchronous/asynchronous distributed systems
- Failure models: faults of processes & communication channels
 - omission and byzantine failures